RoboCab · TSEA29 · Grupp 14

Jimi Henricsson, testansvarig

Automatiska tester och CI

Så fungerar det, och så jobbar vi med det



Varför

Hitta felet vid push, inte på banan

Utan tester

Bilen beter sig konstigt på banan.

Är det koden, kablaget eller en sensor?

Felet kan komma från en ändring någon gjorde för länge sedan.

Med tester

Pipelinen blir röd direkt efter din push.

Loggen visar vilket test och vilken rad det gäller.

Du kan testa logik utan bilen och utan att vänta på någon annan.

Begrepp

Fyra ord att kunna

Test

En funktion som anropar vår kod med känd indata och jämför med facit.

CI

Continuous integration: allt som pushas byggs och testas automatiskt.

Pipeline

Stegen som körs vid en push. De beskrivs i filen .gitlab-ci.yml.

Runner

Datorn som kör stegen. Just nu är det min dator, inte en server.

Hur det fungerar

Det här händer vid varje push

Du pushar
Till vilken branch som helst.

→

GitLab
Skapar en pipeline och väntar på en runner.

→

Runnern
Hämtar koden i en ren container.

→

`build-avr`
Kompilerar för ATmega1284.

→

`unit-tests`
Kör alla tester som har kod att testa.

Grönt: båda stegen gick igenom.

Rött: loggen visar vad som gick fel.

Alla tester körs varje gång. Ingen behöver välja vilka.

Hur det fungerar

Ett test är en vanlig funktion

```
static void test_paket_med_fel_footer_kastas(void)
{
    protocol_parser p;
    protocol_init(&p);

    CHECK_EQ(FEED(&p, H, 1, 42, 0x00), 0);
}
```

1. Förbered

Skapa en ny parser att testa.

2. Mata in känd indata

Ett paket där footern är fel: 0x00.

3. Jämför med facit

Väntat resultat: 0 giltiga paket.

Namnet säger vad som gick fel när testet blir rött.

Vad som testas

Logik testas i CI, hårdvara för hand

Automatiskt, i pipelinen

- UART-paket: giltiga och trasiga
- Kortaste väg och ankomstriktning
- Beslut: stanna, köra igen, styra
- Prioritering av hinderlarm
- PD-beräkning och odometri

För hand, på bilen

- Motor, servo och PWM-signaler
- Rakkörning och svängradie
- Sensorernas noggrannhet och brus
- Wi-Fi: räckvidd och fördröjning
- Hela uppdrag på testbanan

Vad som testas

Läget just nu: 73 tester, 10 körs

Del	Tester	Ansvarig	Status
UART-parser	10	Alla	Körs
PD-reglering	11	Sebastian	Väntar på koden
Odometri	11	Max	Väntar på koden
Ruttplanering	18	Emil	Väntar på koden
Meddelandekö	8	Colin	Väntar på koden
Beslutslogik	8	Colin	Väntar på koden
Lokalisering	7	Colin	Väntar på koden

De 63 väntande testerna är skrivna utifrån designspecen och provade mot tillfällig kod.

Vad som testas

Testerna väntar på er kod

Idag

Pipelinen är ändå grön.

```
VÄNTAR test_pd (implementationen saknas)
```

Du skriver koden

pd.h beskriver vad den ska göra.

```
SteeringModule/logic/pd.c
```

Från den pushen

Testet körs, och fel kod ger rött.

```
OK test_p_del_ar_kp_ganger_fel
```

Hur vi jobbar

Kör testerna på din egen dator

```
# En gång, i Ubuntu (WSL)
sudo apt update
sudo apt install -y build-essential

# Från repots mapp, varje gång
cd /mnt/c/.../robocab
make test
```

Windows

Använd Ubuntu i WSL. Git Bash har inte gcc och make.

Mac och Linux

Det räcker med make och en C-kompilator.

Rätt resultat

Rader med **OK** och **VÄNTAR**, men ingen med **FEL**.

Hur vi jobbar

Ditt arbetsflöde i sex steg

1. Skapa en branch

`git switch -c steering-pd`

Jobba aldrig direkt på **main**.

2. Skriv koden

`<Modul>/logic/`

Logik här, och tester för det du lägger till.

3. Testa lokalt

`make test`

Då ser du felen direkt, utan att vänta på pipelinen.

4. Pusha

`git push -u origin HEAD`

Pipelinen startar automatiskt.

5. Merge request

`din branch → main`

Pipelinens resultat syns där.

6. Merga när det är grönt

Gärna efter att någon annan har läst koden.

Hur vi jobbar

Var koden ska ligga

`<Modul>/logic/`

Logik utan hårdvara, till exempel **pd.c**. Testas i CI. Får inte inkludera **avr/io.h**.

`<Modul>/`

Det som rör hårdvaran: timers, PWM och avbrott. Testas på bilen.

`Common/logic/`

Kod som minst två moduler använder, som UART-protokollet. Byggs även för ATmega, så inga trådar eller sockets.

`Tests/test_<namn>.c`

Testerna. **test_pd.c** hör ihop med **pd.c**. Makefilen hittar nya filer själv.

Hur vi jobbar

När pipelinen blir röd

```
Tests/test_protocol.c:67: FEED(&p, H, 1, 42, 0x00) blev 1, väntade 0
FEL test_paket_med_fel_footer_kastas
```

1. Läs loggen

Under Build → Pipelines, i det röda jobbet.

2. Kör lokalt

make test visar samma fel på din dator.

3. Hitta orsaken

Oftast koden. Är kravet fel: ändra testet och skriv varför.

4. Pusha igen

Till samma branch. En ny pipeline startar.

Ta inte bort ett test bara för att få grönt.

Grönt betyder inte att bilen klarar uppdraget

Inte täckt av testerna än

Kameran och bildbehandlingen · avståndssensorns kalibrering · trådarna i kommunikationsmodulen · spill som bara sker med 16-bitars int på ATmega · all hårdvara

Framåt

Beslut vi behöver ta

Fråga	Förslag i testerna	Vem
Bytevärden i UART-paket	Header 0xAA, footer 0x55, högst 16 databyte	Colin, Sebastian
Namn och argument	Som i headerfilerna	Ansvarig per del
Full meddelandekö	Hinderlarm kastar äldsta lågprioriterade	Colin
$\Delta T \leq 0$ i regulatorn	D-delen blir 0	Sebastian
Kanter i vägnätet	Oriktade, avsnitt 4.7 säger båda delarna	Emil
Tecken på styrning	Positivt värde betyder höger	Colin, Sebastian
Avståndssensor	Inget: specen anger två olika modeller	Max

Framåt

Nästa steg

Vad	Vem
Hämta in main i era brancher och kör make test	Alla
Skriv koden mot headerfilerna och testerna	Ansvarig per del
Sätt upp en andra runner	[namn]
Kräv grön pipeline och skydda main	Jimi, efter runnern

Frågor?